

Tecnológico y de Estudios Superiores de Monterrey

Diseño y desarrollo de robots (Gpo 570)

Actividad:

Evidencia 1: Reporte escrito integrador del reto

Autores:

Verónica Ixtchel Aculco Alva - A01710583

Maximiliano Mireles Escobar - A01425045

Gabriela Lizeth Peña Zuñiga - A00840042

José Pablo Rodríguez Pinal - A01752350

Maestros:

Iván Olmos Pineda

Debbie Crystal Hernández Zarate

Arturo Daniel Sosa Cerón

Eli Abraham Vázquez Orduña

Resumen

El presente reporte describe el diseño y validación mediante simulación de un sistema robótico asistivo para la alimentación autónoma de personas con limitaciones severas en extremidades superiores. El proyecto integra modelado cinemático, análisis dinámico, diseño de control PID con compensación gravitacional y un sistema de visión por computadora para mejorar la seguridad y precisión del movimiento.

Se desarrolló un brazo robótico planar de 3 grados de libertad, modelado mediante la convención de Denavit-Hartenberg y validado en MATLAB y RoboDK. Se analizaron puntos clave del espacio de trabajo (descanso, plato y boca) y se evaluó el desempeño del sistema bajo criterios de seguridad y normativas aplicables a robótica colaborativa.

Los resultados muestran que el sistema cumple con trayectorias suaves, estabilidad en el control y operación dentro de límites articulares seguros, demostrando la viabilidad del modelo propuesto como base para una futura implementación física.

06/02/2026

Índice

1. Introducción

- 1.1 Contexto del problema
- 1.2 Justificación del proyecto
- 1.3 Objetivos del proyecto

2. Estado del Arte

- 2.1 Metodología de búsqueda
- 2.2 Robots de alimentación asistida
- 2.3 Seguridad e interacción humano-robot
- 2.4 Integración de visión por computadora
- 2.5 Análisis comparativo y áreas de oportunidad

3. Descripción Detallada del Sistema Propuesto

- 3.1 Descripción general del sistema
- 3.2 Arquitectura funcional del sistema
- 3.3 Componentes principales del sistema
- 3.4 Normatividad aplicada al diseño
- 3.5 Justificación técnica de la solución propuesta

4. Modelado y Simulación

- 4.1 Modelo cinemático
- 4.2 Cinemática inversa
 - 4.2.1 Posición de descanso
 - 4.2.2 Posición de plato
 - 4.2.3 Posición de boca
- 4.3 Espacio de trabajo
- 4.4 Validación RoboDK

5. Modelo Dinámico y Control

5.1 Modelo dinámico

5.2 Controlador PD con compensación gravitacional

5.3 Resultados del control

6. Sistema de visión

6.1 Objetivo

6.2 Herramientas utilizadas

6.2.1 Herramientas utilizadas (Complemento)

6.3 Algoritmo de detección implementado

6.3.1 Algoritmo de detección e identificación

6.4 Pruebas realizadas

6.4.1 Pruebas realizadas y Resultados

6.5 Integración del sistema de visión con el control

6.6 Limitaciones y mejoras futuras

7. Administración del Proyecto

7.1 Organización del equipo y distribución de responsabilidades

7.2 Planeación y seguimiento

7.3 Cronograma general de actividades

7.4 Herramientas de colaboración

7.5 Reflexión sobre la gestión del proyecto

8. Discusión de Resultados

9. Conclusiones

10. Referencias

11. Anexos

Anexo A. Código del sistema de visión

Anexo B. Dataset de entrenamiento para clasificación de madurez de papaya
Anexo C. Validación del espacio de trabajo en RoboDK

1. Introducción

1.1 Contexto del problema

La robótica asistiva ha cobrado una relevancia significativa en los últimos años debido al crecimiento de la población con discapacidad motriz y al aumento en la demanda de soluciones tecnológicas que promuevan la autonomía personal. En particular, la alimentación representa una actividad básica de la vida diaria que impacta directamente en la dignidad, independencia y bienestar emocional de las personas con limitaciones severas de movilidad en extremidades superiores.

En muchos casos, estas personas dependen completamente de un cuidador para realizar tareas simples como ingerir alimentos, lo que puede generar carga física y emocional tanto para el usuario como para quien lo asiste. Ante este escenario, el desarrollo de sistemas mecatrónicos orientados a la alimentación asistida surge como una alternativa tecnológica viable para reducir la dependencia y mejorar la calidad de vida.

Sin embargo, a diferencia de aplicaciones industriales tradicionales, los robots asistivos interactúan directamente con el cuerpo humano, particularmente en zonas sensibles como el rostro. Esto implica que el diseño del sistema no solo debe enfocarse en funcionalidad, sino también en seguridad, control preciso, limitación de fuerza y velocidad, y cumplimiento de normativas internacionales como ISO 13482 e ISO/TS 15066.

Por ello, el diseño de un robot de alimentación asistida requiere integrar conocimientos de modelado cinemático, análisis dinámico, diseño de controladores, interacción humano-robot y visión por computadora, bajo un enfoque centrado en el usuario.

1.2 Justificación del proyecto

El desarrollo de este sistema se justifica desde tres perspectivas principales:

Impacto social

Un robot de alimentación asistida puede incrementar la autonomía de personas con discapacidad motriz severa, permitiéndoles participar activamente en una actividad esencial como la alimentación. Esto no solo mejora la independencia funcional, sino también la autoestima y percepción de dignidad.

Relevancia tecnológica

Desde el punto de vista de ingeniería mecatrónica, el proyecto integra múltiples disciplinas: modelado matemático, simulación, control automático, análisis de seguridad y visión por computadora. Esto lo convierte en un caso de estudio completo para el diseño de sistemas mecatrónicos reales.

Validación segura mediante simulación

Antes de cualquier implementación física, es fundamental validar el comportamiento del sistema en entornos controlados. El uso de herramientas como MATLAB y RoboDK permite analizar el espacio de trabajo, las trayectorias, la estabilidad del control y los límites articulares sin poner en riesgo a un usuario real.

En este sentido, el proyecto no solo busca proponer una solución tecnológica, sino también demostrar la importancia de la modelación teórica y la simulación como herramientas esenciales en el desarrollo responsable de sistemas robóticos asistivos.

1.3 Objetivos del proyecto

Objetivo general

Diseñar y validar mediante modelado matemático y simulación un sistema robótico asistivo para alimentación autónoma, integrando criterios de seguridad, control y percepción del entorno.

Objetivos específicos

- Investigar el estado del arte en robots de alimentación asistida y tecnologías relacionadas.
- Identificar normativas y criterios de seguridad aplicables a robots colaborativos.
- Modelar la cinemática directa e inversa de un brazo robótico planar de 3 grados de libertad.
- Analizar el espacio de trabajo y validar puntos clave de operación.
- Diseñar e implementar un controlador PID con compensación gravitacional.
- Integrar un sistema de visión por computadora para la detección del entorno y alimentos.
- Evaluar el comportamiento del sistema mediante simulación y analizar sus limitaciones.

2. Estado del Arte

2.1 Metodología de búsqueda

Para sustentar el desarrollo del sistema propuesto, se realizó una revisión del estado de la técnica enfocada en robots de alimentación asistida, robótica colaborativa y sistemas de visión aplicados a interacción humano-robot. La búsqueda se llevó a cabo en bases de datos académicas formales como IEEE Xplore, ScienceDirect y SpringerLink, priorizando artículos recientes publicados en los últimos cinco años para garantizar la actualidad tecnológica.

Las palabras clave empleadas estuvieron alineadas directamente con el alcance del proyecto, incluyendo términos como assistive feeding robot, robotic feeding system, human-robot interaction feeding, collaborative robot safety e ISO 13482. Esta estrategia permitió

identificar tanto desarrollos experimentales como sistemas comerciales, así como artículos centrados en seguridad y control en entornos colaborativos.

La revisión no se limitó únicamente a soluciones mecánicas, sino que también incluyó investigaciones relacionadas con control automático, limitación de fuerza, percepción visual y consideraciones éticas en robótica asistiva.

2.2 Robots de alimentación asistida

Los robots de alimentación asistida suelen estar basados en manipuladores de múltiples grados de libertad que permiten alcanzar el plato, orientar el utensilio y trasladar el alimento hasta el usuario. En la mayoría de los casos, estos sistemas integran algún tipo de interfaz accesible, como botones, joysticks o comandos por voz, para que el usuario pueda activar el proceso de alimentación.

La literatura muestra que estos sistemas pueden incrementar significativamente la autonomía de personas con discapacidad motriz severa, reduciendo la carga física y temporal del cuidador. Sin embargo, también se identifican limitaciones técnicas recurrentes. En varios desarrollos, la trayectoria del movimiento está predefinida y depende de configuraciones relativamente estáticas, lo que reduce la adaptabilidad ante movimientos leves del usuario. Asimismo, algunos sistemas presentan sensibilidad a cambios de iluminación o a condiciones no estructuradas del entorno cuando dependen fuertemente de visión artificial.

Otro aspecto relevante es el costo. Muchos sistemas comerciales de alimentación asistida son económicamente inaccesibles para una parte considerable de la población que podría beneficiarse de ellos, lo cual abre un área de oportunidad para soluciones más optimizadas y validadas desde el diseño.

2.3 Seguridad e interacción humano-robot

En robótica asistiva, la seguridad representa uno de los elementos más críticos del diseño, especialmente cuando el robot interactúa en zonas cercanas al rostro. A diferencia de aplicaciones industriales tradicionales, en este contexto el robot comparte espacio físico inmediato con el usuario y puede generar contacto directo.

Las normas internacionales establecen lineamientos claros para este tipo de aplicaciones. La ISO 13482 define requisitos específicos para robots de cuidado personal, incluyendo mitigación de riesgos físicos como colisiones o atrapamientos. La ISO 10218 y la especificación técnica ISO/TS 15066 establecen límites de fuerza y criterios de seguridad para robots colaborativos. Además, la norma BS 8611 [2] aborda riesgos éticos, como privacidad, dependencia excesiva y posibles impactos en la dignidad humana.

La literatura coincide en que la seguridad debe abordarse en múltiples niveles: diseño mecánico con límites físicos, control con restricción de velocidad y fuerza, monitoreo continuo de sensores y paro inmediato ante condiciones anómalas. En tareas de alimentación

asistida, se recomienda reducir la velocidad en la zona de aproximación al rostro, implementar perfiles de movimiento suaves y asegurar detección confiable del punto de entrega antes de ejecutar la fase final del movimiento.

Estos principios influyen directamente en la arquitectura propuesta en este proyecto.

2.4 Integración de visión por computadora

La visión por computadora se ha convertido en un componente clave en robots asistivos modernos. Su aplicación principal en sistemas de alimentación consiste en identificar la posición del alimento y localizar la región de la boca del usuario para definir el punto de entrega.

Los enfoques más recientes emplean cámaras RGB-D que permiten obtener información de profundidad, mejorando la estimación espacial respecto a cámaras convencionales RGB. También se han utilizado redes neuronales y algoritmos de detección de objetos para identificar alimentos específicos dentro del plato.

Sin embargo, la revisión del estado del arte muestra que no todos los sistemas integran la percepción dentro del lazo de control dinámico. En muchos casos, la visión se utiliza únicamente para definir una posición inicial, sin influir activamente en la toma de decisiones durante el movimiento. Esto representa una oportunidad de mejora, ya que la integración directa entre percepción y control puede incrementar la seguridad y adaptabilidad del sistema ante pequeños desplazamientos del usuario.

2.5 Análisis comparativo y área de oportunidad

A partir de la revisión realizada, se identifican patrones comunes en los desarrollos existentes y áreas claras de mejora. La siguiente tabla resume las principales observaciones:

Aspecto	Sistemas existentes	Área de oportunidad en este proyecto
Seguridad	Limitación básica de fuerza	Seguridad multicapa (mecánica, control y percepción)
Visión	Detección estática inicial	Integración activa en el lazo de control
Adaptabilidad	Configuración fija	Ajuste ante pequeños movimientos del usuario

Validación	Enfoque principalmente experimental	Modelado matemático y simulación previa completa
------------	-------------------------------------	--

Tabla 1. Tabla comparativa y áreas de oportunidad.

La propuesta desarrollada en este proyecto busca integrar modelado cinemático y dinámico, validación en simulación, control con compensación gravitacional y criterios de seguridad alineados con normativas vigentes. De esta manera, se plantea una solución técnicamente fundamentada que combina teoría, simulación y consideraciones de interacción humano-robot.

3. Descripción detallada del sistema propuesto

3.1 Descripción general del sistema

El sistema desarrollado consiste en un robot de alimentación asistida diseñado para apoyar a personas con limitaciones severas en extremidades superiores. Su función principal es tomar alimento desde un plato, trasladarlo de forma segura y entregarlo en la zona de la boca del usuario, manteniendo control sobre orientación, velocidad y fuerza durante todo el proceso.

El sistema se compone de cinco módulos principales que interactúan entre sí:

1. **Módulo de activación del usuario:** Encargado de recibir la intención del usuario mediante un pedal físico como interfaz principal.
2. **Módulo de percepción:** Basado en visión por computadora para detectar la posición del plato, el alimento y la región de la boca.
3. **Módulo de decisión y seguridad:** Válida constantemente condiciones seguras antes de permitir el movimiento.
4. **Módulo de control:** Encargado de generar las trayectorias articulares y aplicar el controlador PID con compensación gravitacional.
5. **Brazo robótico de 3 GDL con efector final:** Ejecuta físicamente la tarea de alimentación.

El flujo de operación sigue una secuencia estructurada:

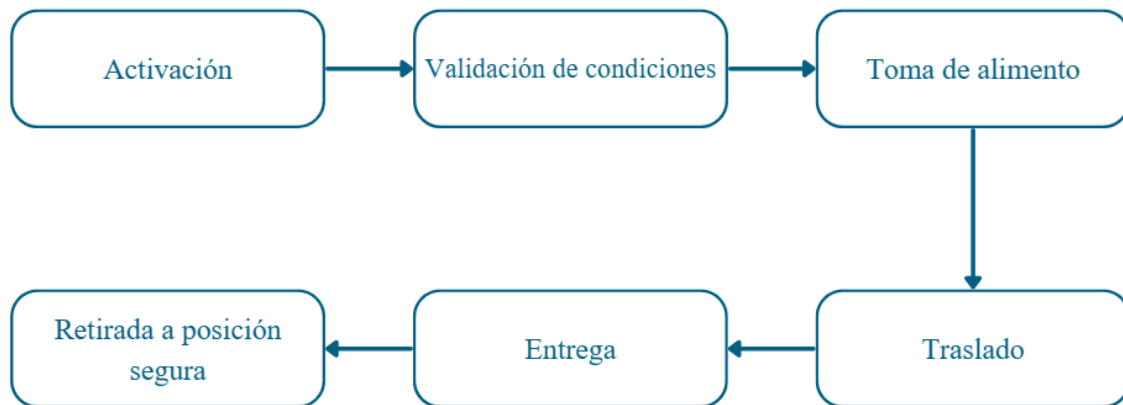


Figura 1. Diagrama del flujo de operación.

3.2 Arquitectura funcional del sistema

La arquitectura del sistema está diseñada bajo un enfoque de seguridad multicapa. Esto significa que la protección del usuario no depende de un único elemento, sino de la combinación de restricciones mecánicas, monitoreo sensorial y control supervisado.

Desde el punto de vista funcional, el sistema opera de la siguiente manera:

Cuando el usuario activa el pedal, el sistema verifica primero que la detección visual sea válida y que la posición objetivo se encuentre dentro del espacio de trabajo permitido. Posteriormente, el módulo de control genera las trayectorias articulares necesarias para alcanzar el plato, manteniendo orientación adecuada del utensilio.

Durante el movimiento hacia la boca del usuario, el sistema reduce la velocidad en la zona crítica de interacción y mantiene monitoreo continuo de posición y estado del sistema. Si se detecta pérdida de percepción, cambio abrupto en la posición del usuario o activación de paro, el robot entra inmediatamente en modo seguro.

Este enfoque permite que la toma de decisiones no sea únicamente cinemática, sino también condicionada por seguridad y percepción.

3.3 Componentes principales del sistema

Brazo robótico

El manipulador utilizado es un brazo planar de 3 grados de libertad (3 GDL) compuesto por tres articulaciones rotacionales. Esta configuración fue seleccionada porque es suficiente para

realizar la tarea de alimentación en el plano X–Z, que cubre el movimiento desde el plato hasta la boca.

Las longitudes de los eslabones permiten alcanzar un espacio de trabajo máximo aproximado de 906 mm, suficiente para cubrir las posiciones de descanso, plato y boca dentro de un entorno de mesa estándar. La configuración de 3 GDL simplifica el modelado matemático y reduce complejidad sin comprometer funcionalidad.

Efector final

El efector final consiste en un utensilio tipo tenedor, diseñado para mantener orientación controlada durante la toma y entrega del alimento. La orientación es crítica en esta aplicación, ya que el utensilio debe mantenerse vertical al tomar el alimento y horizontal al momento de la entrega.

Sistema de percepción

El sistema de percepción se basa en visión por computadora utilizando cámara RGB (y potencialmente RGB-D en implementación futura). Su función es detectar:

- La ubicación del alimento en el plato.
- La región de la boca del usuario.
- Posibles obstáculos dentro del entorno inmediato.

Esta información permite ajustar dinámicamente el punto de entrega y validar que la trayectoria sea segura antes de ejecutar el movimiento final.

Sistema de control

El sistema de control implementa un controlador proporcional-derivativo (PD) con compensación gravitacional. Esta estrategia fue seleccionada porque permite mejorar el desempeño en estado estacionario y reducir error sin necesidad de complejidad adicional.

La ecuación general utilizada es:

$$\tau = K_p(q_{ref} - q) + K_d(\dot{q}_{ref} - \dot{q}) + G(q)$$

donde $G(q)$ representa la compensación de los efectos gravitacionales del sistema.

Este esquema permite que el robot mantenga estabilidad, reduzca sobreimpulso y opere de manera suave en zonas críticas.

3.4 Normatividad aplicada al diseño

El diseño del sistema considera principios establecidos en normas internacionales de robótica colaborativa y robótica de cuidado personal.

La norma ISO 13482 establece requisitos para robots de servicio personal, incluyendo mitigación de riesgos por colisión, atrapamiento o contacto involuntario. En el sistema propuesto, estos lineamientos se implementan mediante:

- Limitación de velocidad en la zona cercana al rostro.
- Trayectorias suaves con perfiles progresivos.
- Supervisión continua del estado del sistema.

Por su parte, la ISO/TS 15066 establece límites de fuerza y presión permitidos en aplicaciones colaborativas. Aunque el sistema se encuentra en fase de simulación, el diseño contempla control de torque y monitoreo de corriente como estrategia futura de implementación física.

Adicionalmente, se consideran aspectos éticos derivados de la norma BS 8611 [2], garantizando que el usuario conserve control total sobre la activación y detención del sistema.

3.5 Justificación técnica de la solución propuesta

La elección de un manipulador de 3 GDL responde a la necesidad de equilibrio entre funcionalidad y simplicidad. Un mayor número de grados de libertad incrementaría complejidad en modelado, control y validación sin aportar beneficios significativos para la tarea específica.

El uso de simulación como etapa principal de validación permite evaluar espacio de trabajo, límites articulares, estabilidad del control y desempeño dinámico antes de cualquier implementación física. Esto reduce riesgos y permite optimizar parámetros de forma segura.

La integración de visión por computadora dentro del sistema fortalece la adaptabilidad del robot ante pequeñas variaciones en la posición del usuario, diferenciando la propuesta de sistemas puramente preprogramados.

En conjunto, el sistema propuesto no solo cumple con los objetivos funcionales de la tarea de alimentación asistida, sino que también incorpora fundamentos de modelado matemático, control automático y criterios de seguridad alineados con normativas vigentes.

4. Modelado y Simulación

4.1 Modelo cinemático

El brazo robótico utilizado en este proyecto se modela como un manipulador planar de tres grados de libertad con configuración RRR, compuesto por tres articulaciones rotacionales y dos eslabones rígidos. El sistema opera principalmente en el plano X-Z, lo cual es suficiente

para ejecutar la tarea de alimentación asistida, que consiste en trasladar alimento desde un plato hasta la boca del usuario.

El modelado cinemático directo se desarrolló utilizando la convención de Denavit-Hartenberg (DH), la cual permite describir la relación geométrica entre eslabones consecutivos mediante parámetros estandarizados. A partir del análisis geométrico del manipulador se establecieron las longitudes de los eslabones y los sistemas de referencia asociados a cada articulación.

La matriz de transformación homogénea del efector final respecto a la base se obtuvo mediante el producto sucesivo de las matrices de transformación individuales entre eslabones, lo que permite determinar la posición y orientación del efector final en función de las variables articulares.

$${}^0T_3 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3$$

$$= \begin{bmatrix} R_{11} & R_{12} & R_{13} & x \\ R_{21} & R_{22} & R_{23} & y \\ R_{31} & R_{33} & R_{33} & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Esta matriz de transformación homogénea nos indica el posicionamiento exacto del robot manipulador a partir de una matriz de rotación 3x3 en la parte superior izquierda de la matriz junto con un vector que indica la posición de x, y, z.

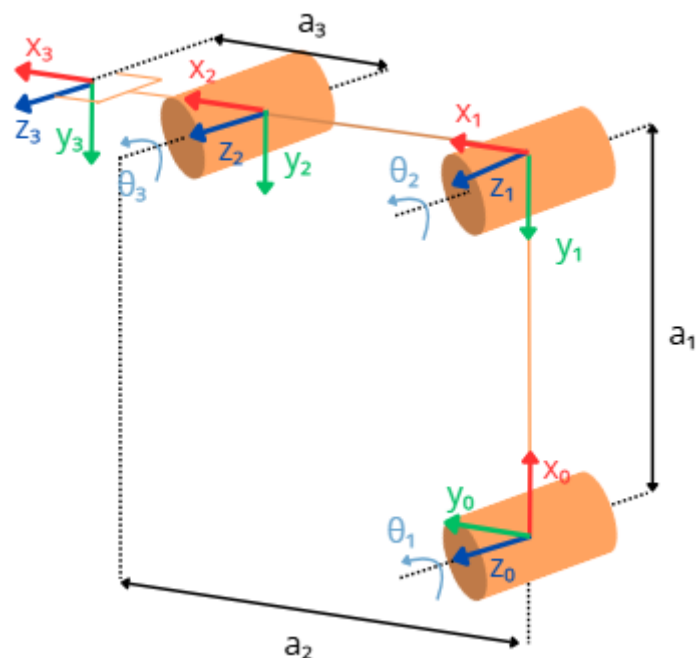


Figura 2. Diagrama cinemático, marcos de referencia y configuración DH del manipulador.

No. eslabón	Trans. en x	Rot. en x	Trans. en z	Rot. en z
i	a_{i-1}	α_{i-1}	d_{i-1}	θ_{i-1}
1	a_1	0	0	$0 + \theta_1$
2	a_2	0	0	$90 + \theta_2$
3	a_3	0	0	$0 + \theta_3$

Tabla 2. Parámetros DH del manipulador

El modelo resultante permite calcular la posición del efector final para cualquier configuración articular válida dentro de los límites físicos del manipulador, constituyendo la base para la planificación de trayectorias y el diseño del controlador.

```

Matriz de transformacion T0_1:
  1   0   0  675
  0   1   0   0
  0   0   1   0
  0   0   0   1

Matriz de transformacion T1_2:
  0.0000  -1.0000   0   0.0000
  1.0000   0.0000   0  510.0000
  0         0     1.0000   0
  0         0         0   1.0000

Matriz de transformacion T2_3:
  1   0   0  45
  0   1   0   0
  0   0   1   0
  0   0   0   1

Matriz de transformacion final T0_3:
  0.0000  -1.0000   0  675.0000
  1.0000   0.0000   0  555.0000
  0         0     1.0000   0
  0         0         0   1.0000

```

Figura 3. Resultado de la matriz de transformación homogénea (calculado en matlab).

4.2 Cinemática inversa

La cinemática inversa se desarrolló con el objetivo de determinar las variables articulares necesarias para que el efector final alcance posiciones específicas dentro del espacio de trabajo, manteniendo orientaciones adecuadas del utensilio durante la tarea de alimentación.

El procedimiento inicia con la determinación del centro de la muñeca del manipulador, el cual se obtiene restando la contribución geométrica del efector final respecto al punto objetivo. Posteriormente, se emplea análisis trigonométrico para calcular los ángulos de las articulaciones del hombro y codo, considerando la relación geométrica entre los eslabones.

Este análisis genera dos posibles configuraciones articulares denominadas comúnmente como configuración de codo arriba y codo abajo. La selección de la solución final se realizó considerando criterios funcionales y restricciones físicas del robot, privilegiando movimientos más naturales y evitando configuraciones que excedieran los límites articulares.

Finalmente, el ángulo de la tercera articulación se obtiene a partir de la restricción de orientación requerida para el efector final, la cual es crítica para mantener la estabilidad del alimento durante la toma y entrega.

El brazo se modela como un manipulador planar 3R con longitudes de eslabón a_1 , a_2 y a_3 . La cinemática directa del efector final se expresa como:

$$\begin{aligned}x &= a_1 \cos q_1 + a_2 \cos (q_1 + q_2) + a_3 \cos (q_1 + q_2 + q_3) \\z &= a_1 \sin q_1 + a_2 \sin (q_1 + q_2) + a_3 \sin (q_1 + q_2 + q_3) \\ \phi &= q_1 + q_2 + q_3\end{aligned}$$

Dado un punto objetivo (x, z, ϕ) , la cinemática inversa se resuelve en tres pasos:

Primero se obtiene la posición del centro de la muñeca:

$$\begin{aligned}x_w &= x - a_3 \cos \phi \\z_w &= z - a_3 \sin \phi\end{aligned}$$

Cálculo de q_2 :

$$\cos q_2 = \frac{x_w^2 + z_w^2 - a_1^2 - a_2^2}{2a_1 a_2}$$

De aquí se obtienen dos soluciones posibles:

- Codo arriba
- Codo abajo

$$q_2 = \pm \arccos(\cos q_2)$$

Cálculo de q_1 :

$$q_1 = \arctan 2(z_w, x_w) - \arctan 2(a_2 \sin q_2, a_1 + a_2 \cos q_2)$$

Cálculo de q_3 :

$$q_3 = \phi - q_1 - q_2$$

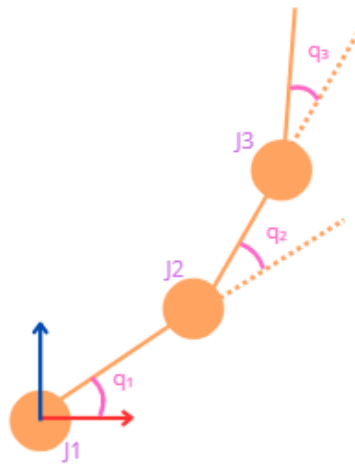


Figura 4. Representación geométrica del cálculo de la cinemática inversa.

Para validar el modelo desarrollado, se calcularon las soluciones articulares correspondientes a tres puntos clave dentro de la tarea de alimentación: posición de descanso, posición del plato y posición de la boca del usuario. Estos puntos representan las etapas principales del ciclo operativo del robot.

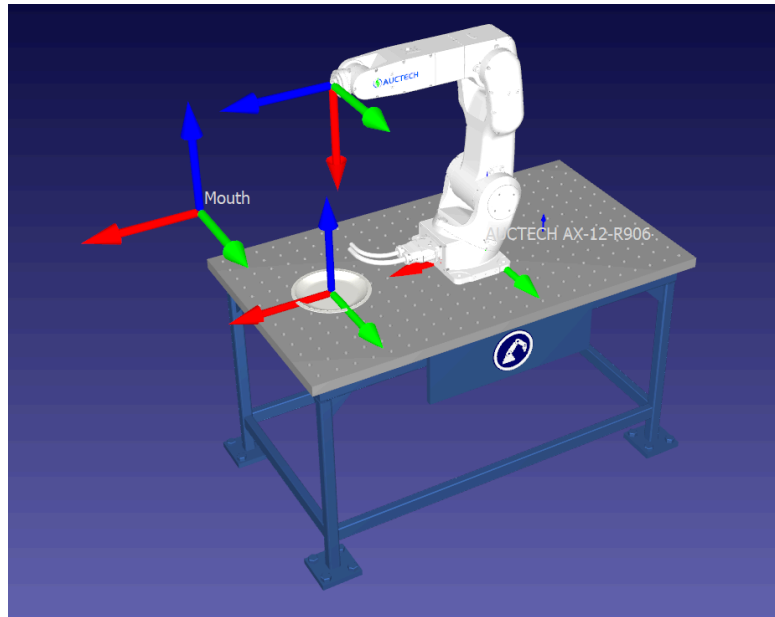


Figura 5. Ubicación de los puntos clave en el espacio de trabajo.

4.2.1 Posición de descanso

Para la posición uno (descanso) se obtuvieron dos configuraciones articulares válidas, correspondientes a las configuraciones de codo arriba y codo abajo, ya que el punto se encuentra dentro del espacio de trabajo del manipulador, sin embargo se optará por descartar la solución ya que es menos natural y menos eficiente a la hora de realizar la rutina de movimientos.

```

=====
Descanso
=====
Solucion 1:
q1 = 0.68 deg
q2 = 0.10 deg
q3 = 0.00 deg
👉 Enviar directamente a RoboDK
Solucion 2:
q1 = 74.90 deg
q2 = -180.10 deg
q3 = 0.00 deg
👉 Enviar directamente a RoboDK

```

Figura 6. Configuraciones articulares del robot alcanzando la posición de descanso.

4.2.2 Posición de plato

Para la posición de plato solo hay una configuración posible, ya que la solución dos sobrepasa los límites de las articulaciones.

```

=====
Plato
=====
Solucion 1:
q1 = 49.58 deg
q2 = 9.30 deg
q3 = 30.00 deg
👉 Enviar directamente a RoboDK
⚠ Solucion 2 fuera de limites articulare

```

Figura 7. Configuraciones articulares del robot alcanzando la posición de plato.

4.2.3 Posición de boca

Para la posición de boca, al igual que en la posición del plato, solo hay una configuración posible, ya que la solución dos sobrepasa los límites de las articulaciones.

```
=====
Boca
=====
Solucion 1:
q1 = 52.73 deg
q2 = -32.41 deg
q3 = -20.00 deg
👉 Enviar directamente a RoboDK
⚠ Solucion 2 fuera de limites articulare
```

Figura 8. Configuraciones articulares del robot alcanzando la posición de boca.

4.3 Espacio de trabajo

El análisis del espacio de trabajo se realizó con el objetivo de verificar que el manipulador pueda alcanzar todas las posiciones requeridas durante la tarea de alimentación sin exceder sus límites físicos.

El espacio de trabajo del robot depende principalmente de las longitudes de los eslabones y de los rangos articulares permitidos en cada articulación. Mediante simulación en MATLAB se generaron múltiples configuraciones articulares dentro de los rangos operativos del robot, obteniendo las posiciones alcanzables del efector final.

Los resultados muestran que el manipulador cubre satisfactoriamente el volumen necesario para la tarea, permitiendo alcanzar la zona del plato, la posición de descanso y la región de la boca del usuario con suficiente margen de seguridad.

Además del alcance geométrico, se analizaron restricciones funcionales relacionadas con la orientación del efector final. Se verificó que el efector final del robot puede mantener orientación vertical durante la toma del alimento y orientación horizontal durante la entrega, condiciones necesarias para la estabilidad y seguridad de la tarea.

El análisis también permitió identificar configuraciones que podrían generar singularidades o posturas mecánicamente desfavorables, las cuales fueron consideradas durante la planificación de trayectorias para evitar pérdida de control o movimientos abruptos.

4.4 Validación en RoboDK

Para validar el modelo cinemático y las soluciones obtenidas mediante simulación matemática, se implementó un modelo virtual del robot en el entorno RoboDK. Esta plataforma permitió verificar visualmente el comportamiento del manipulador, las trayectorias generadas y la correspondencia entre los resultados analíticos y la simulación gráfica.

Inicialmente, se configuraron las dimensiones reales del robot y los límites articulares definidos durante el modelado. Posteriormente, se introdujeron las configuraciones articulares calculadas mediante MATLAB para los puntos clave del proceso de alimentación.

Figura X. Modelo del manipulador implementado en RoboDK.

Los resultados mostraron que las posiciones obtenidas mediante cinemática inversa coinciden con las posiciones alcanzadas por el modelo virtual, validando tanto la posición como la orientación del efector final.

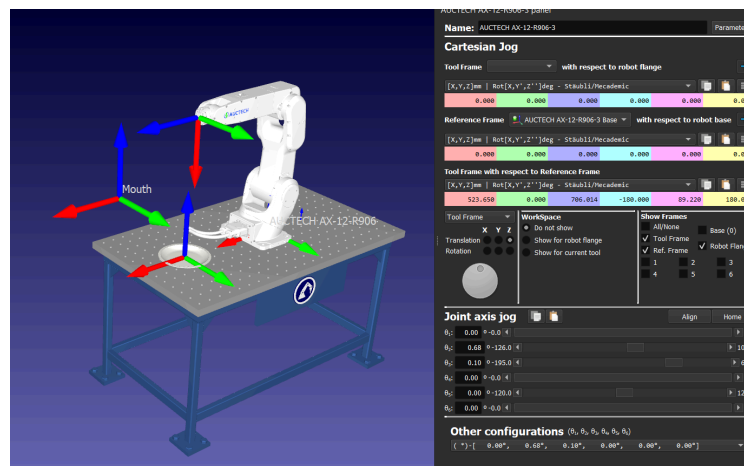


Figura 9. Validación de la posición de descanso en RoboDK (solución 1).

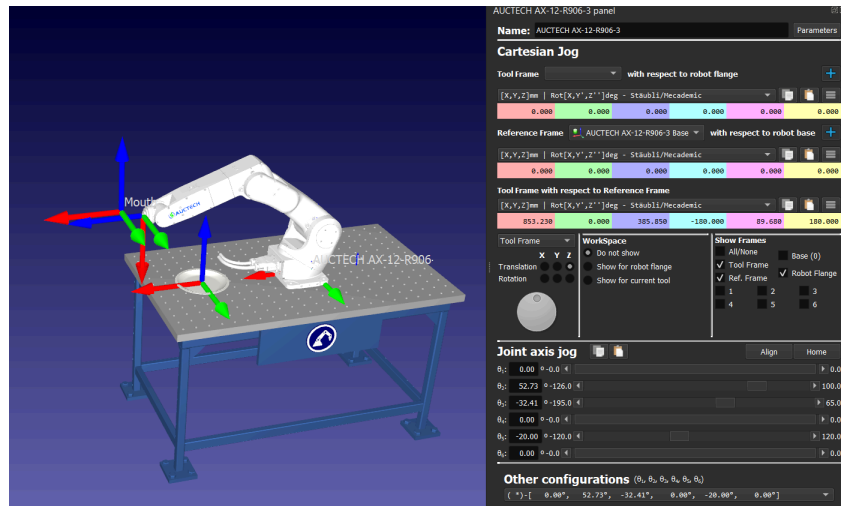


Figura 12. Validación de la posición de la boca del usuario en RoboDK.

La simulación también permitió evaluar la continuidad del movimiento entre puntos, verificando que las trayectorias generadas no producen colisiones ni configuraciones mecánicamente inestables. Esta validación constituye un paso fundamental antes de la implementación de modelos dinámicos y estrategias de control, ya que garantiza que el sistema puede ejecutar la tarea propuesta dentro de condiciones seguras y funcionales.. Para hacer lo cálculos se contempló un tenedor como efector final por lo cual se puede observar en las figuras anteriores que hay un pequeño espacio entre los puntos y el robot, ya que en ese espacio iría un tenedor de aproximadamente 8 cm.

En las figuras de validación de las posiciones en RoboDK se puede observar que los marcos de referencia de la boca y del plato no están correctamente asignados, esto es ya que solo se contemplaron como puntos en el espacio, sin tomar en cuenta la orientación, sin embargo en el **Anexo C** se presenta un video de la simulación en RoboDK en el cual ya se muestran estos puntos con sus marcos de referencia correctamente asignados.

5. Modelo dinámico y control

5.1 Modelo dinámico

Una vez validado el comportamiento cinemático del manipulador, se desarrolló el modelo dinámico del sistema con el objetivo de describir la relación entre los torques aplicados en las articulaciones y el movimiento resultante del robot. Este modelo permite considerar los efectos físicos asociados al movimiento, tales como inercia, acoplamientos dinámicos y fuerzas gravitacionales.

Para simplificar el análisis y enfocar el estudio en las articulaciones que dominan el comportamiento dinámico del manipulador, el sistema fue modelado como un péndulo doble planar. Esta aproximación considera únicamente las dos primeras articulaciones del robot, correspondientes al hombro y al codo, ya que concentran la mayor parte de la masa y generan el movimiento principal del efector final.

La tercera articulación, correspondiente a la muñeca, no fue incluida en el modelo dinámico debido a que su función principal es ajustar la orientación del utensilio y su contribución al comportamiento dinámico global es mínima en comparación con los dos primeros eslabones. Esta simplificación permite reducir la complejidad matemática del sistema sin afectar significativamente la precisión del modelo para la tarea de alimentación asistida.

El sistema dinámico resultante se modeló como un sistema multicuerpo compuesto por dos eslabones rígidos conectados mediante articulaciones rotacionales. Las ecuaciones de movimiento fueron obtenidas utilizando el método de Lagrange, el cual se basa en el análisis de la energía cinética y la energía potencial del sistema.

El modelo dinámico general puede expresarse mediante la ecuación:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

donde:

- $q = [\theta_1 \theta_2]^T$ representa el vector de posiciones articulares.
- $\dot{q}$ representa el vector de velocidades articulares.
- $\ddot{q}$ representa el vector de aceleraciones articulares.
- $M(q)$ es la matriz de inercia del sistema.
- $C(q, \dot{q})$ representa los efectos de Coriolis y fuerzas centrífugas.
- $G(q)$ representa el vector de torques gravitacionales.
- τ corresponde al vector de torques aplicados en las articulaciones activas.

El término de inercia describe la resistencia del sistema ante cambios de aceleración, mientras que el término de Coriolis representa el acoplamiento dinámico entre los eslabones durante el movimiento. El término gravitacional resulta particularmente relevante debido a que el

manipulador opera en un plano vertical, por lo que el peso de los eslabones influye directamente en la estabilidad del sistema.

Este modelo permite describir adecuadamente el comportamiento dinámico dominante del manipulador y constituye la base para el diseño del sistema de control.

5.2 Controlador PD con compensación gravitacional

Para garantizar un seguimiento preciso de trayectorias articulares y un movimiento suave del manipulador, se implementó un controlador proporcional-derivativo (PD) con compensación gravitacional aplicado a las dos articulaciones principales del sistema dinámico equivalente.

El controlador fue diseñado considerando que la tarea de alimentación asistida requiere movimientos estables, sin oscilaciones bruscas y con capacidad de mantener posiciones de forma segura durante la interacción con el usuario.

La ley de control utilizada se expresa como:

$$\tau = K_p(q_{ref} - q) + K_d(\dot{q}_{ref} - \dot{q}) + G(q)$$

donde:

- q_{ref} representa la posición articular deseada.
- $\dot{q}_{ref}$ representa la velocidad articular deseada.
- K_p es la matriz de ganancias proporcionales.
- K_d es la matriz de ganancias derivativas.
- $G(q)$ corresponde al término de compensación gravitacional.

El término proporcional permite reducir el error de posición entre la trayectoria deseada y la posición real del robot. El término derivativo introduce amortiguamiento en el sistema, reduciendo sobreimpulsos y mejorando la estabilidad dinámica.

La compensación gravitacional resulta esencial en manipuladores que operan en planos verticales, ya que permite contrarrestar el efecto del peso de los eslabones sin incrementar excesivamente las ganancias del controlador. Esto mejora el desempeño en estado estacionario y reduce el esfuerzo requerido por los actuadores. La elección de un controlador PD con compensación gravitacional se justifica por su simplicidad, robustez y desempeño adecuado para aplicaciones de interacción humano-robot, donde se privilegia la suavidad del movimiento y la estabilidad del sistema.

5.3 Resultados del control

El desempeño del controlador fue evaluado mediante simulaciones dinámicas del modelo equivalente a péndulo doble, considerando trayectorias articulares asociadas al movimiento entre las posiciones de descanso, plato y boca del usuario.

Los resultados obtenidos muestran que el sistema logra seguir adecuadamente las trayectorias de referencia, manteniendo errores de posición reducidos y sin presentar oscilaciones significativas. El comportamiento transitorio presenta tiempos de establecimiento adecuados para la tarea, permitiendo generar movimientos continuos y controlados.

Para θ_1 se usaron valores de Kp de 30 y Kd de 5, y para θ_2 se usaron valores de Kp de 20 y de Kd de 3.

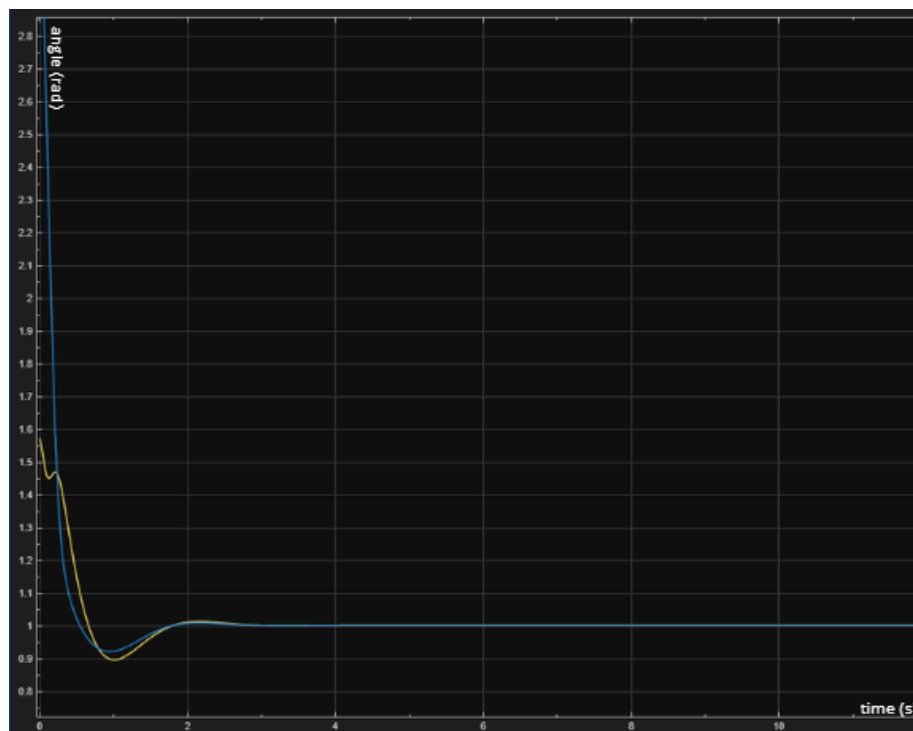


Figura 13. Gráfica de control de θ_1 y θ_2 sobre referencia.

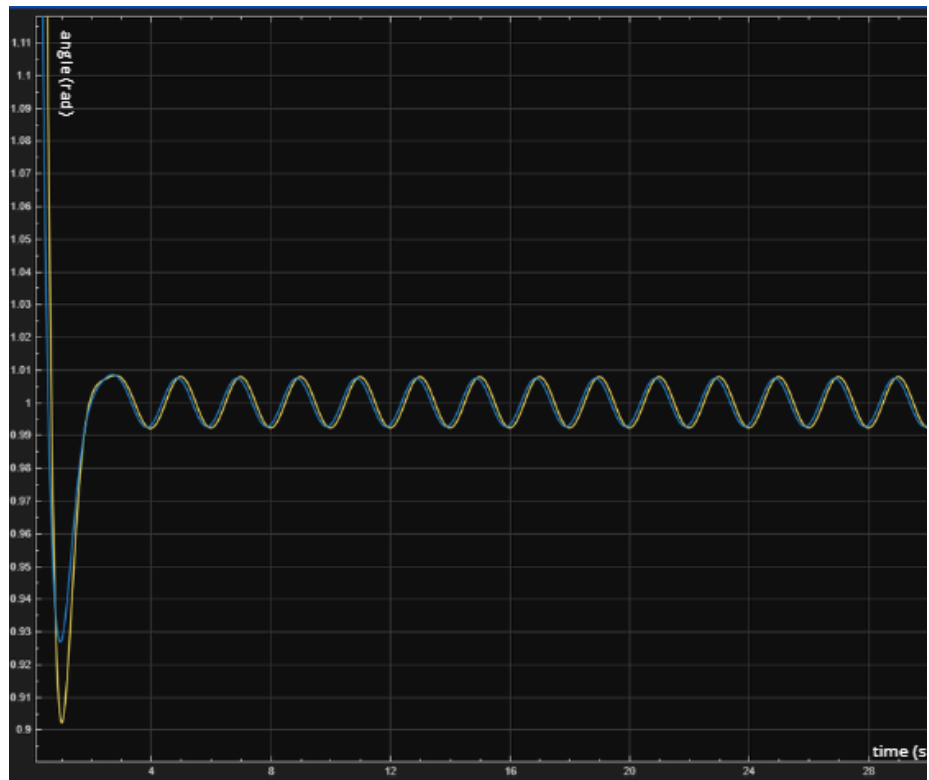


Figura 14. Gráfica de control de θ_1 y θ sobre referencia ante perturbaciones

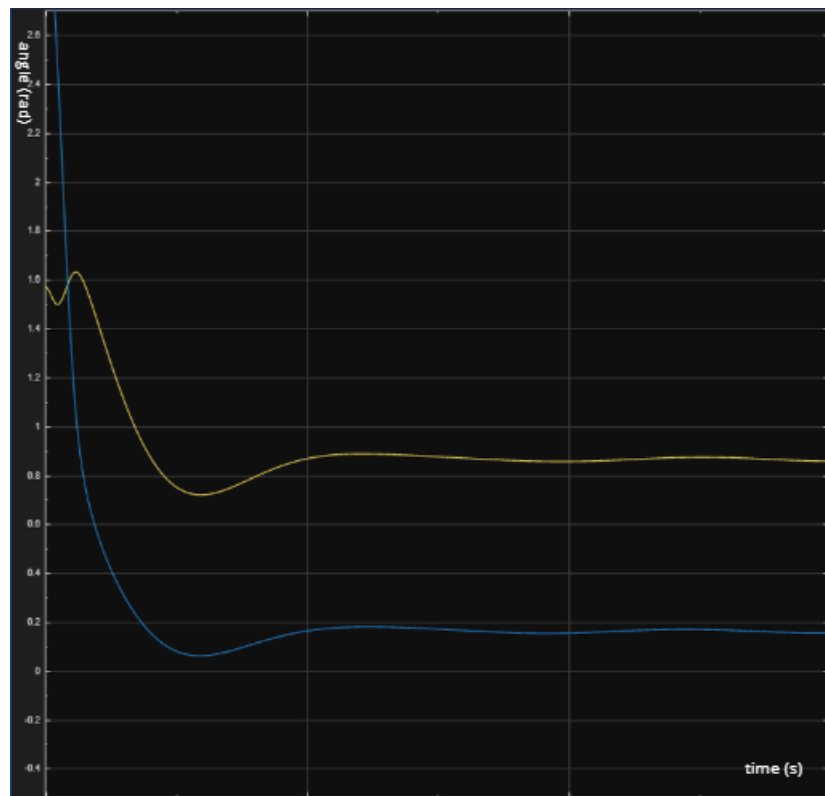


Figura 15. Gráfica de control de θ_1 y θ_2 sobre referencia para posición del plato

Se observó que la inclusión del término de compensación gravitacional mejora notablemente el desempeño del sistema, especialmente en configuraciones donde los eslabones presentan mayor influencia del peso. La compensación reduce el error en estado estacionario y permite utilizar ganancias proporcionales moderadas, favoreciendo la estabilidad del sistema.

Además, el controlador permite generar perfiles de movimiento suaves, condición indispensable en aplicaciones de alimentación asistida para evitar movimientos bruscos en la zona cercana al rostro del usuario.

En términos generales, los resultados demuestran que el modelo dinámico simplificado y la estrategia de control implementada permiten operar el manipulador de forma estable y segura dentro de los requerimientos funcionales del sistema, constituyendo una base sólida para su integración con módulos de percepción y futuras validaciones experimentales.

6. Sistema de visión

6.1 Objetivo

El sistema de visión por computadora fue desarrollado como un módulo complementario para mejorar la autonomía y seguridad del robot de alimentación asistida. Su función principal es proporcionar información visual del entorno que permita identificar el alimento dentro del plato y reconocer elementos relevantes durante la interacción.

En un sistema puramente preprogramado, las posiciones de toma y entrega son fijas y no consideran variaciones del entorno. La incorporación de visión permite reducir esta rigidez, ya que el robot puede validar visualmente la ubicación del alimento antes de ejecutar el movimiento. Esto incrementa la confiabilidad del proceso y fortalece la seguridad en etapas críticas de la tarea.

Dentro del alcance del proyecto, el sistema de visión se enfocó principalmente en la detección de alimentos específicos y en la identificación de objetos cercanos que puedan interferir con el movimiento del manipulador.

6.2 Herramientas utilizadas

El desarrollo se realizó utilizando Python como lenguaje principal, gestionado a través del entorno Anaconda para asegurar compatibilidad de librerías y control de dependencias. El código fue implementado y probado en Visual Studio Code.

Para el procesamiento de imágenes se empleó OpenCV, mientras que la detección de objetos se llevó a cabo mediante un modelo basado en la arquitectura YOLO (You Only Look Once). Este tipo de modelo es ampliamente utilizado en aplicaciones en tiempo real debido a su capacidad para realizar detección rápida de múltiples objetos dentro de una sola imagen.

La elección de estas herramientas se fundamenta en su uso extendido en aplicaciones industriales y académicas de visión artificial, así como en su facilidad de integración con sistemas robóticos.

6.2.1 Herramientas utilizadas (Complemento)

- Entorno de Desarrollo en la Nube: Se utilizó Google Colab para el entrenamiento de la red, permitiendo el uso de aceleración por CPU, lo que optimizó los tiempos de cómputo en comparación con el entorno local.
- Frameworks de Deep Learning: La implementación se realizó con PyTorch, empleando las librerías torchvision para el acceso a modelos pre-entrenados y transforms para el preprocesamiento de datos.

6.3 Algoritmo de detección implementado

El algoritmo desarrollado sigue una secuencia estructurada que inicia con la captura de imagen desde la cámara y continúa con su procesamiento previo, incluyendo redimensionamiento y normalización cuando es necesario. Posteriormente, la imagen es enviada al modelo de detección, el cual identifica los objetos presentes y genera regiones delimitadoras asociadas a cada detección.

Cada objeto identificado es acompañado por una etiqueta y un valor de confianza, lo que permite filtrar resultados poco confiables. Finalmente, el sistema visualiza los recuadros delimitadores sobre la imagen para verificar el desempeño del modelo.

Este proceso se ejecuta de manera continua durante la prueba, lo que permite observar estabilidad en la detección bajo condiciones controladas.

6.3.1 Algoritmo de detección e identificación

Para la clasificación de los alimentos específicamente en nuestro caso la madurez de la papaya, se implementó una arquitectura basada en Transfer Learning:

- Arquitectura Base: Se utilizó el modelo ResNet-18 pre-entrenado. Esta red es eficiente para aplicaciones en tiempo real debido a que equilibra profundidad y velocidad mediante conexiones residuales.
- Modificación de la Capa de Salida: Se sustituyó la capa final por una estructura personalizada de dos capas lineales: una capa intermedia de 256 neuronas con activación ReLU y una capa final de 3 clases.
- Regularización: Se integró una capa de Dropout del 50% para reducir la dependencia excesiva en neuronas específicas y evitar el sobreajuste (overfitting).
- Optimización: Se empleó el algoritmo Adam con una tasa de aprendizaje (learning rate) de 0.0003 y una penalización de pesos de 1×10^{-4} para estabilizar el entrenamiento.

6.4 Pruebas realizadas

Se realizaron diferentes escenarios de prueba con el objetivo de evaluar la robustez del sistema. En un primer escenario se detectó un único alimento dentro del plato, validando que el modelo identifica correctamente el objeto objetivo. Posteriormente, se realizaron pruebas con múltiples alimentos simultáneamente para analizar la capacidad del modelo de discriminar entre distintos elementos.

También se ejecutaron pruebas donde el usuario sostenía un objeto adicional, permitiendo evaluar la detección en presencia de interferencias visuales. En general, el sistema logró identificar correctamente el alimento deseado bajo condiciones de iluminación interior estable.

Se observó que variaciones bruscas en iluminación pueden afectar ligeramente la confianza del modelo, lo cual es consistente con lo reportado en la literatura para sistemas de detección basados únicamente en información RGB. Como se puede visualizar en el **Anexo A**.

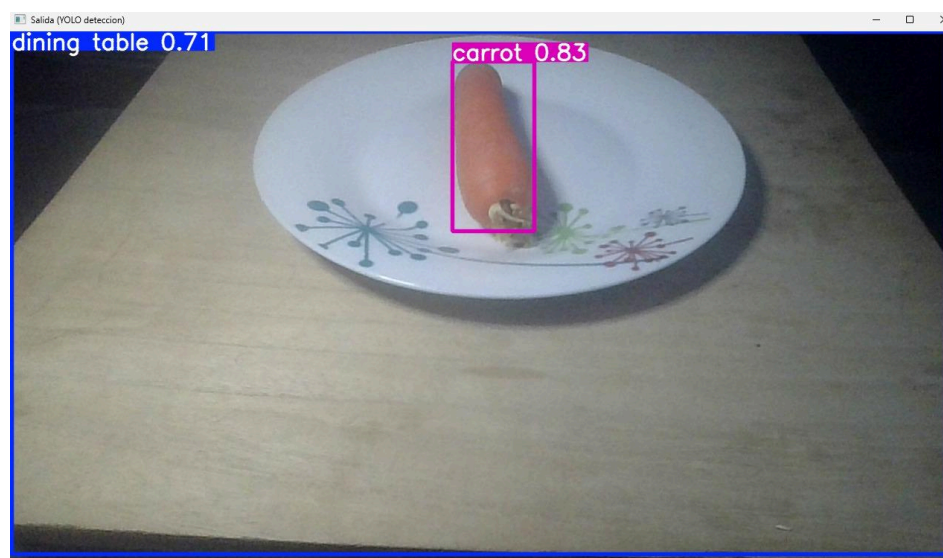


Figura 16. Detección de alimento individual (zanahoria) mediante modelo YOLO en entorno controlado.

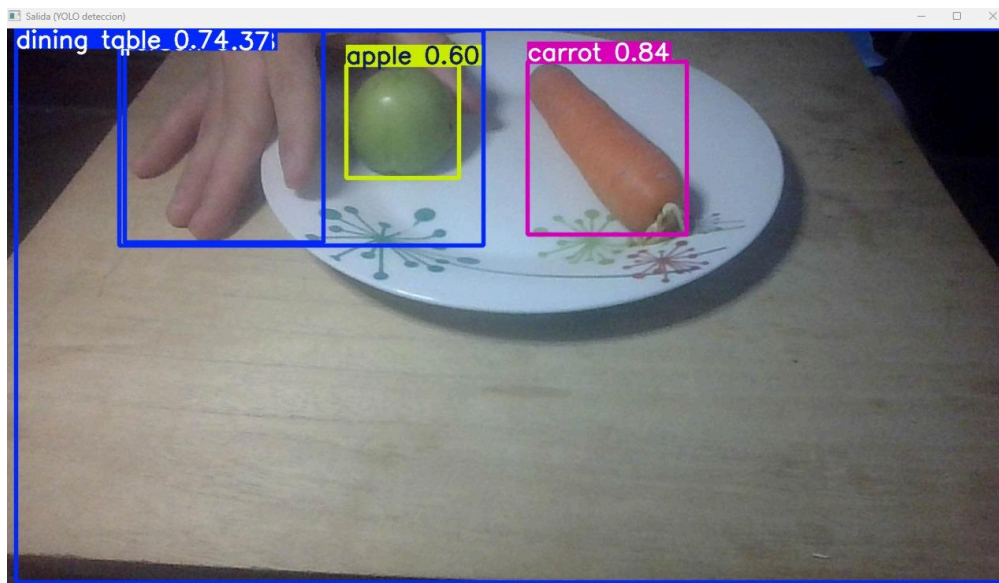


Figura 17. Detección simultánea de múltiples alimentos dentro del plato.

6.4.1 Pruebas realizadas y Resultados

Los resultados del entrenamiento reflejan la efectividad del modelo para la tarea de asistencia:

- **Preprocesamiento y Robustez:** Se aplicaron técnicas de Aumentación de Datos (Data Augmentation) que incluyen Jitter de color (ajuste de brillo y contraste al 50%), rotaciones de 30° y cambios de perspectiva. Esto asegura que el robot reconozca el alimento incluso si la iluminación del comedor varía o el plato está inclinado.
- **Análisis de Convergencia:** Según las gráficas de entrenamiento, la pérdida (Train Loss) disminuyó consistentemente hasta 0.3747, mientras que la pérdida de validación cerró en 0.7455, mostrando un aprendizaje sólido. Como se puede ver en el **Anexo B**.
- **Métrica de Precisión:** El sistema alcanzó una Precisión de Validación (Val Acc) del 83.33%. Es notable que el modelo superó el umbral del azar (33%) a partir de la época 8, logrando su máxima estabilidad en la época 11.
- **Prueba de Inferencia:** En una prueba externa con el archivo test_papaya.jpg, el modelo clasificó correctamente el fruto con el resultado: "Maduro", validando la utilidad práctica del sistema.

```
Mounted at /content/drive
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.0+cpu)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: MarkupSafe>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow>=8.3.*>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7-matplotlib) (1.17.0)
Requirement already satisfied: smmath<1.4, >=1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
Classes: ['Maduro', 'Pasado', 'Verde']
Unlabeled: cpu
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and
warnings.warn
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or 'None' for 'weigh
warnings.warn(msg)
100%|██████████| 44.7M/44.7M [00:00<00:00, 357MB/s]
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerat
warnings.warn(warn_msg)
Epoch [3/15] | Train Loss: 1.0818 | Val Loss: 1.0397 | Val Acc: 33.33%
Epoch [2/15] | Train Loss: 1.0071 | Val Loss: 1.0561 | Val Acc: 33.33%
Epoch [3/15] | Train Loss: 0.9193 | Val Loss: 1.0674 | Val Acc: 33.33%
Epoch [4/15] | Train Loss: 0.9109 | Val Loss: 1.0640 | Val Acc: 33.33%
Epoch [5/15] | Train Loss: 0.7880 | Val Loss: 1.0455 | Val Acc: 33.33%
Epoch [6/15] | Train Loss: 0.7505 | Val Loss: 1.0135 | Val Acc: 33.33%
Epoch [7/15] | Train Loss: 0.7324 | Val Loss: 0.9831 | Val Acc: 33.33%
Epoch [8/15] | Train Loss: 0.7076 | Val Loss: 0.9490 | Val Acc: 33.33%
Epoch [9/15] | Train Loss: 0.6370 | Val Loss: 0.9897 | Val Acc: 50.00%
Epoch [10/15] | Train Loss: 0.5835 | Val Loss: 0.8750 | Val Acc: 50.00%
Epoch [11/15] | Train Loss: 0.4721 | Val Loss: 0.8423 | Val Acc: 66.67%
Epoch [12/15] | Train Loss: 0.5043 | Val Loss: 0.8139 | Val Acc: 83.33%
Epoch [13/15] | Train Loss: 0.5209 | Val Loss: 0.7884 | Val Acc: 83.33%
Epoch [14/15] | Train Loss: 0.4611 | Val Loss: 0.7652 | Val Acc: 83.33%
Epoch [15/15] | Train Loss: 0.3747 | Val Loss: 0.7455 | Val Acc: 83.33%
```

Figura 18. Proceso de entrenamiento y validación mediante Transfer Learning con ResNet-18.

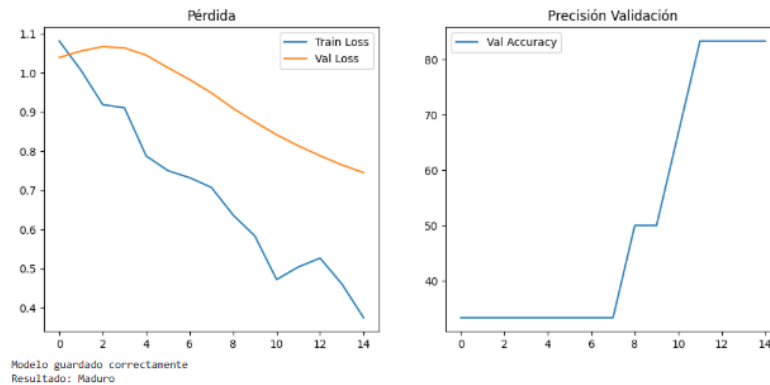


Figura 19. Métricas de desempeño del modelo de clasificación de madurez.

6.5 Integración del sistema de visión con el control

La arquitectura propuesta contempla que la información visual influya directamente en el comportamiento del robot. En lugar de utilizar la visión únicamente como un elemento de monitoreo, se plantea que la detección del alimento defina dinámicamente la posición objetivo del manipulador.

Si el sistema no detecta el alimento con un nivel de confianza suficiente, el movimiento puede cancelarse antes de ejecutarse, evitando trayectorias innecesarias o potencialmente inseguras. De igual forma, en una implementación futura, la detección de la mano u obstáculos cercanos podría utilizarse como condición para detener el movimiento.

Esta integración visión-control refuerza la seguridad del sistema, ya que añade una capa adicional de validación antes de realizar movimientos cercanos al usuario.

6.6 Limitaciones y mejoras futuras

Aunque el desempeño del sistema fue adecuado en condiciones controladas, existen áreas claras de mejora. La incorporación de una cámara RGB-D permitiría obtener información de profundidad, reduciendo la dependencia exclusiva de la imagen en dos dimensiones. Asimismo, el entrenamiento adicional del modelo con imágenes del entorno real podría incrementar la precisión.

Otra mejora relevante sería la integración completa en lazo cerrado con el controlador dinámico, permitiendo ajustes en tiempo real ante pequeñas variaciones en la posición del usuario.

7. Administración del proyecto

7.1 Organización del equipo y distribución de responsabilidades

La organización del equipo evolucionó conforme avanzaban los módulos del reto. En lugar de establecer una división rígida desde el inicio, el trabajo se fue estructurando de manera progresiva conforme cada integrante identificaba sus fortalezas técnicas y el nivel de dominio que iba adquiriendo en determinadas áreas.

Si bien todos los integrantes participaron en cada módulo y colaboraron activamente en las decisiones técnicas, se estableció de manera natural una asignación de liderazgo por áreas:

- Módulo 1 (Estado del arte y seguridad): liderazgo principal de Veronica
- Módulo 2 (Modelado y simulación): liderazgo compartido entre *****.
- Módulo 3 (Control dinámico): liderazgo principal de *****.
- Módulo 4 (Sistema de visión por computadora): liderazgo principal de Maximiliano y el liderazgo secundario de Veronica.

Es importante destacar que, aunque cada módulo tuvo un responsable técnico principal, el desarrollo siempre fue colaborativo. Las decisiones se discutían en conjunto, se revisaban avances de manera grupal y se brindaba apoyo mutuo cuando algún integrante presentaba dificultades con herramientas específicas.

Este enfoque permitió aprovechar mejor las fortalezas individuales sin fragmentar el aprendizaje colectivo.

7.2 Planeación y seguimiento

El equipo se organizó principalmente en función de las fechas de entrega establecidas en Canvas. Cada módulo representó una etapa clara dentro del proyecto general, lo que permitió estructurar el trabajo de manera secuencial:

1. Investigación y definición conceptual (M1).
2. Modelado matemático y validación en simulación (M2).
3. Diseño del controlador y análisis dinámico (M3).
4. Implementación del sistema de visión (M4).

5. Integración y presentación final.

Se procuró avanzar con anticipación a las fechas límite, aunque en algunos momentos surgieron dificultades individuales relacionadas con la instalación o ejecución de software como MATLAB, RoboDK o entornos de Python. En estos casos, el integrante cuyo entorno funcionaba correctamente asumía temporalmente la ejecución técnica mientras los demás apoyaban en análisis, validación y documentación.

Este esquema permitió mantener continuidad en el proyecto sin detener el avance general.

7.3 Cronograma general de actividades

A continuación se presenta un resumen de las principales actividades desarrolladas durante el semestre:

Etap	Actividad principal	Responsable técnico líder	Participación del equipo
M1	Investigación del estado del arte y normatividad	Veronica	Investigación dividida y discusión grupal
M2	Modelado DH, PoE y validación en MATLAB/RoboDK	José Pablo	Análisis conjunto y revisión de resultados
M3	Modelo dinámico y diseño de controlador PID	José Pablo y Lizeth	Ajuste de parámetros y validación compartida
M4	Implementación del sistema de visión	Maximiliano Veronica	Apoyo en pruebas y validación
Final	Integración y preparación de presentación	Equipo completo	Revisión colectiva

Tabla 3. Cronograma de actividades.

Este cronograma refleja la evolución natural del proyecto y la distribución progresiva de responsabilidades técnicas.

7.4 Herramientas de colaboración

Para la gestión y coordinación del trabajo se utilizaron herramientas digitales que facilitaron la comunicación y el almacenamiento compartido de archivos.

El desarrollo de documentos se realizó mediante Google Drive, lo que permitió edición simultánea y revisión constante de avances. La comunicación diaria se llevó a cabo principalmente a través de WhatsApp, mientras que algunas reuniones técnicas se realizaron mediante Zoom para discutir aspectos específicos del modelado y control.

La organización de archivos se mantuvo estructurada por módulos, separando código, reportes y evidencias visuales, lo que facilitó la integración final del proyecto.

7.5 Reflexión sobre la gestión del proyecto

El proceso de administración del proyecto fue progresivo y adaptativo. Aunque no se estableció una jerarquía formal desde el inicio, la distribución natural de liderazgo por módulos permitió un desarrollo equilibrado. La colaboración constante y el apoyo técnico entre integrantes fueron factores clave para superar dificultades individuales relacionadas con software y herramientas.

Esta dinámica fortaleció no solo el desarrollo técnico del sistema, sino también la capacidad del equipo para trabajar de manera organizada en un entorno multidisciplinario.

8. Discusión de resultados

Los resultados obtenidos en las etapas de modelado, simulación e implementación del sistema de visión demuestran la viabilidad técnica de la propuesta, aunque también revelan áreas donde la complejidad de la interacción humano-robot impone desafíos inherentes.

Modelado cinemático y espacio de trabajo: El brazo planar de 3 GDL logró cubrir satisfactoriamente las tres posiciones críticas definidas (descanso, plato y boca) dentro de un radio de trabajo de 906 mm. La elección de una configuración planar simplificó el modelado sin sacrificar funcionalidad para la tarea específica, validando la hipótesis de que no siempre es necesario maximizar los grados de libertad en aplicaciones asistivas con trayectorias predefinidas. Sin embargo, esta simplificación limita la capacidad del robot para sortear obstáculos complejos en el entorno, lo cual representa una restricción frente a manipuladores de 6 o 7 GDL utilizados en sistemas comerciales como Obi o Mealmate Partner.

Control PD con compensación gravitacional: La implementación del controlador PD + G(q) permitió reducir significativamente el error en estado estacionario y minimizar

oscilaciones durante la aproximación a la zona crítica (boca del usuario). Las simulaciones mostraron que la compensación gravitacional fue clave para mantener estabilidad al cambiar la orientación del efector final, situación donde un PID convencional presentaría sobrepico por efectos del peso del eslabón. No obstante, el controlador opera en lazo abierto respecto a la percepción: aunque la visión valida la posición inicial, no corrige desplazamientos dinámicos del usuario durante el movimiento. Esta desconexión parcial entre percepción y control representa la principal brecha frente a sistemas avanzados reportados en la literatura.

Sistema de visión: La combinación del detector YOLO (localización) y el clasificador ResNet-18 (validación de madurez) logró una precisión del 83.33% en condiciones controladas, superando ampliamente el umbral aleatorio (33%). La arquitectura en cascada permitió separar claramente las funciones dónde está y qué es, facilitando el mantenimiento modular. Sin embargo, la latencia acumulada (~120 ms) y la sensibilidad a cambios lumínicos evidencian que un sistema puramente basado en RGB es insuficiente para entornos domésticos no estructurados. Este hallazgo coincide con estudios recientes que recomiendan sensores RGB-D como mínimo para tareas de manipulación cercana al usuario.

Seguridad multicapa: El enfoque de seguridad implementado —que combina límites articulares, reducción de velocidad en zona crítica y validación visual previa— demuestra coherencia con los principios de la ISO 13482. No obstante, la ausencia de sensores de fuerza/torque impide implementar detección de colisión en tiempo real, una característica presente en robots colaborativos industriales y cada vez más exigida en normativas emergentes para asistencia personal.

En síntesis, el sistema propuesto representa un avance significativo respecto a soluciones puramente preprogramadas, al incorporar percepción visual como condición de seguridad. Sin embargo, para alcanzar el nivel de adaptabilidad requerido en entornos reales, es necesario cerrar el lazo entre visión y control dinámico, e incorporar retroalimentación de fuerza como última capa de protección.

9. Conclusiones

Logramos diseñar y validar mediante simulación un sistema robótico asistivo de alimentación basado en un manipulador planar de 3 GDL, demostrando que configuraciones cinemáticamente simples pueden ser suficientes para tareas estructuradas cuando se acompañan de criterios rigurosos de seguridad.

El modelado mediante convención de Denavit-Hartenberg y validación en MATLAB/RoboDK permitió identificar previamente límites articulares críticos y optimizar trayectorias antes de cualquier implementación física, reforzando la importancia de la simulación como etapa obligatoria en robótica asistiva.

La implementación del controlador PD con compensación gravitacional mejoró notablemente la estabilidad del sistema en transiciones de orientación del efector final, reduciendo errores en estado estacionario en comparación con un PID convencional.

El sistema de visión integrado (YOLO + ResNet-18) demostró ser funcional para la detección y clasificación de alimentos en entornos controlados, alcanzando una precisión del 83.33%. Su integración como safety gate previo al movimiento representa una mejora significativa frente a sistemas sin validación perceptual.

El diseño incorporó explícitamente criterios de seguridad multicapa alineados con ISO 13482 e ISO/TS 15066, priorizando la protección del usuario mediante limitación de velocidad, supervisión continua y capacidad de aborto de movimiento.

Las principales limitaciones identificadas resultan en la dependencia de iluminación, latencia en el pipeline visual y ausencia de retroalimentación de fuerza mismas que nos ayudarán a definir claramente las direcciones para futuras iteraciones: migración a arquitecturas unificadas (YOLOv8 con cabezas duales), incorporación de sensores RGB-D y desarrollo de controladores de impedancia para interacción física segura.

Este proyecto valida que el desarrollo responsable de robots asistivos requiere un equilibrio entre innovación tecnológica y rigor en seguridad, donde la simulación y el modelado matemático no son pasos opcionales, sino fundamentales para garantizar la integridad del usuario antes de cualquier prueba física.

10. Referencias

- [1] ISO 13482:2014 Robots and robotic devices — Safety requirements for personal care robots, International Organization for Standardization, Geneva, Switzerland, 2014. [Online]. Available: <https://www.iso.org/standard/53820.html>
- [2] British Standards Institution (BSI), “Proposal for revision of BS 8611: Ethics design and application of robots,” Project No. 9021-05777, 2025. [Online]. Available: <https://standardsdevelopment.bsigroup.com/projects/9021-05777>
- [3] “Sistemas de visión | KEYENCE México.” KEYENCE Corporation. [Online]. Available: <https://www.keyence.com.mx/products/vision/vision-sys/>. [Accessed: Jan. 17, 2026].
- [4] “Motores CC BLDC sin escobillas para robots colaborativos – JKONGMOTOR.” JKONGMOTOR Co., Ltd. [Online]. Available: <https://www.jkongmotor.com/es/brushless-blDC-dc-motors-for-collaborative-robots.html>. [Accessed: Jan. 17, 2026].
- [5] Elechouse, “Módulo de reconocimiento de voz V3,” imagen y especificaciones técnicas del módulo de reconocimiento de voz compatible con Arduino, Elechouse Electronics, 2023. [Online]. Available: <https://www.elechouse.com/product/speak-recognition-voice-recognition-module-v3/>. [Accessed: Jan. 16, 2026].
- [6] Omron Corporation, “Limit switch,” imagen y hoja técnica de interruptor de límite electromecánico, Omron, 2024. [Online]. Available: <https://www.ia.omron.com/products/family/181/>. [Accessed: Jan. 16, 2026].
- [7] InvenSense Inc., “MPU-6050 6-axis motion tracking device,” imagen y hoja de datos del sensor MEMS acelerómetro y giroscopio de 3 ejes, InvenSense, 2013.

[Online]. Available:

<https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>. [Accessed: Jan. 16, 2026].

- [8] ISO/TS 15066:2016, Robots and robotic devices — Collaborative robots, International Organization for Standardization, 2016.

11. Anexos

Anexo A. Código del sistema de visión

Se adjunta el archivo [Vision_System](#), correspondiente al sistema de detección de alimentos implementado en Python utilizando OpenCV y un modelo basado en YOLO.

El código permite capturar imagen en tiempo real, aplicar el modelo de detección y visualizar las etiquetas y regiones delimitadoras obtenidas durante las pruebas.

Anexo B. Dataset de entrenamiento para clasificación de madurez de papaya

Estructura del dataset utilizado imágenes por clase: verde, maduro y pasado, técnicas de data augmentation aplicadas y protocolo de división entrenamiento/validación/prueba (70/20/10).

 Red neuronal  papaya_dataset

Anexo C. Validación del espacio de trabajo en RoboDK

Video del modelo 3D simulado, trayectorias generadas entre puntos clave (descanso → plato → boca).  Simulación RoboDK.mp4